

POLITECNICO
MILANO 1863

Ingegneria del Software

Ingegneria Informatica – Prof. San Pietro

Esercitazione:

Ereditarietà & Interfacce

My Contacts:
Giuseppe Sorrentino
giuseppe.sorrentino@polimi.it

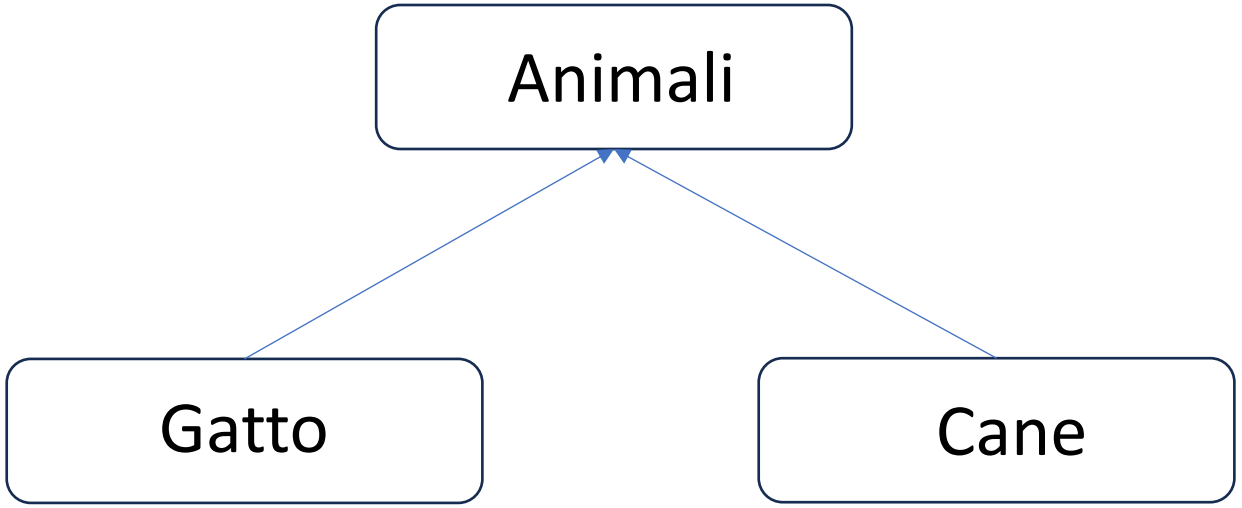
Example: Geometric Shapes

- Si implementi una gerarchia di classe che rappresenti delle forme geometriche.
- In particolare si implementi il calcolo dell'area, del perimetro e la rappresentazione su schermo.

Cosa dovrei usare?

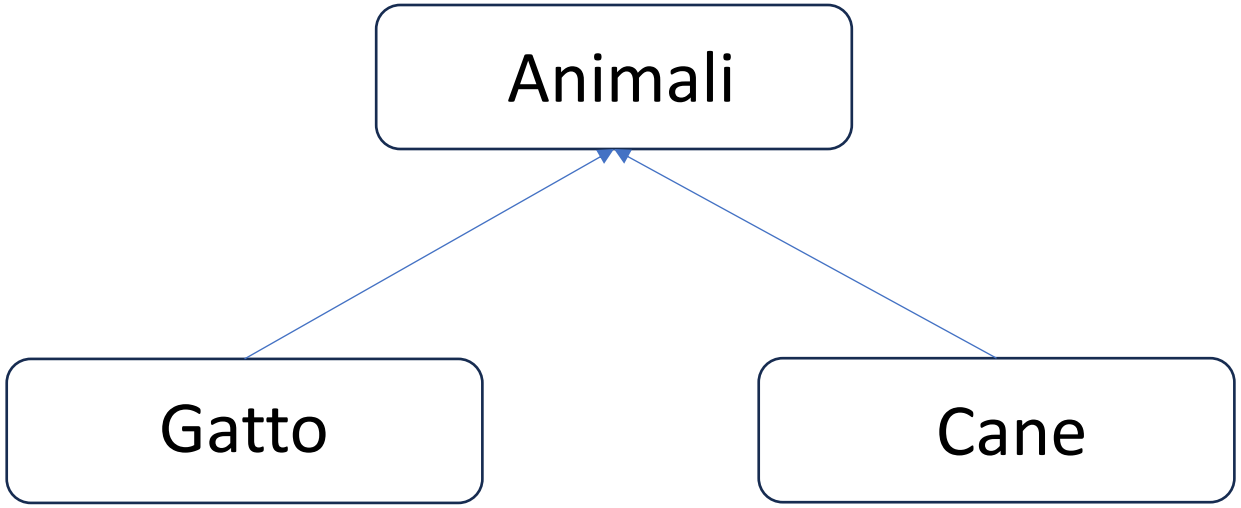
Ereditarietà ed Interfacce

- Consideriamo questa gerarchia



Ereditarietà ed Interfacce

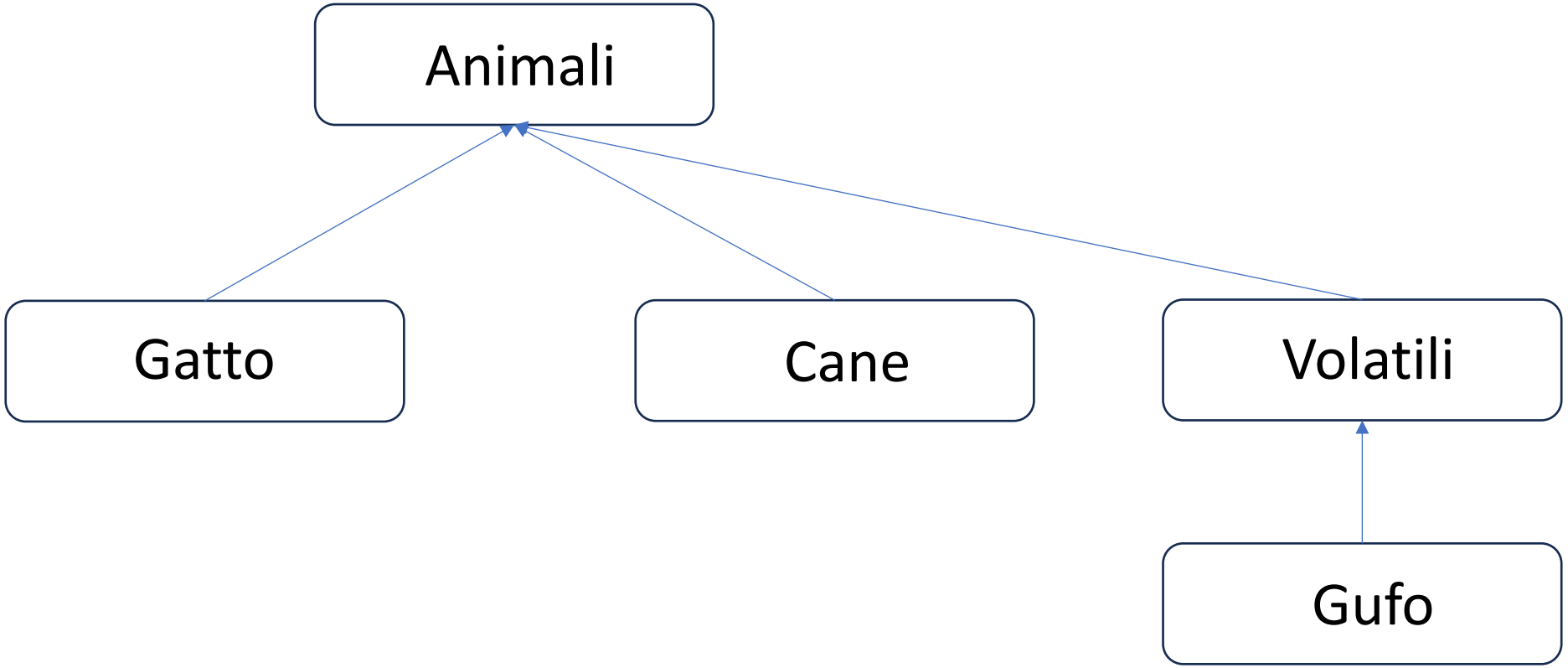
- Consideriamo questa gerarchia



- Come possiamo estendere questa gerarchia per includere volatili?

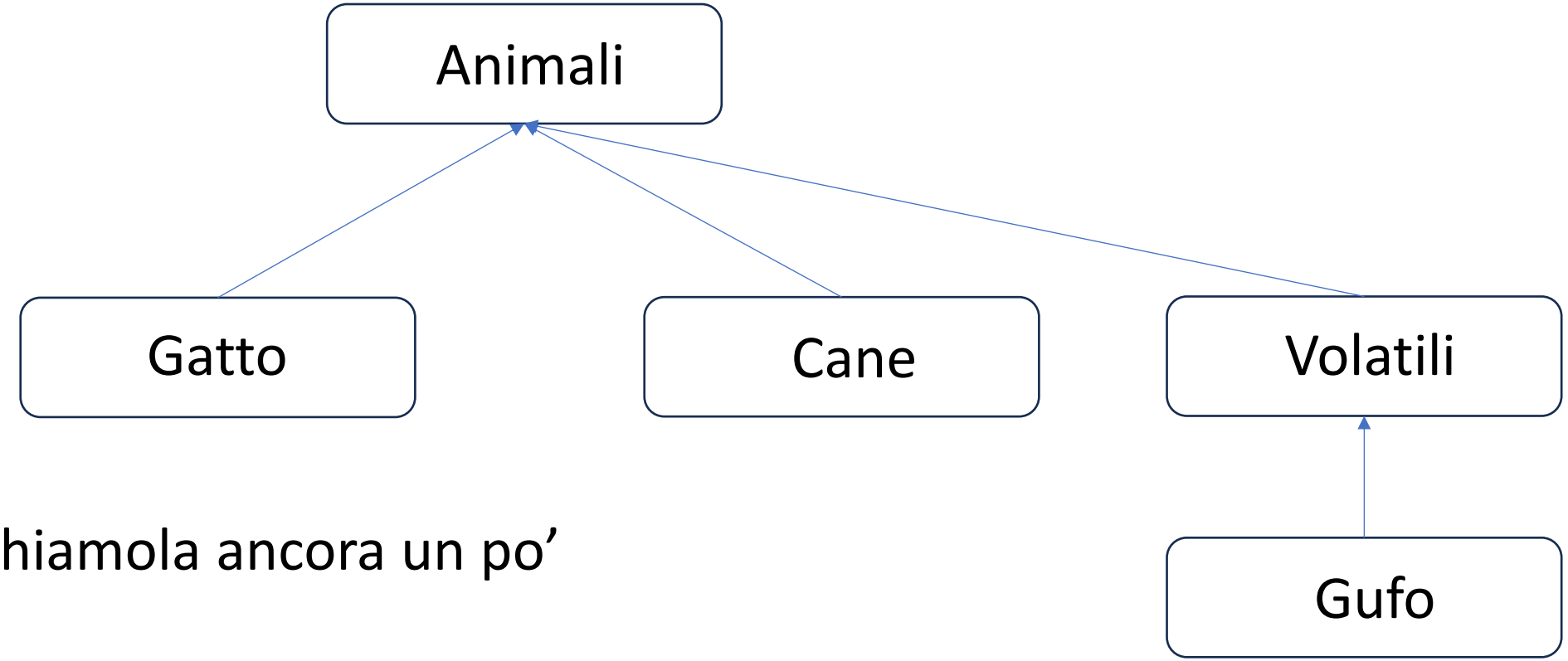
Ereditarietà ed Interfacce

- Consideriamo questa gerarchia



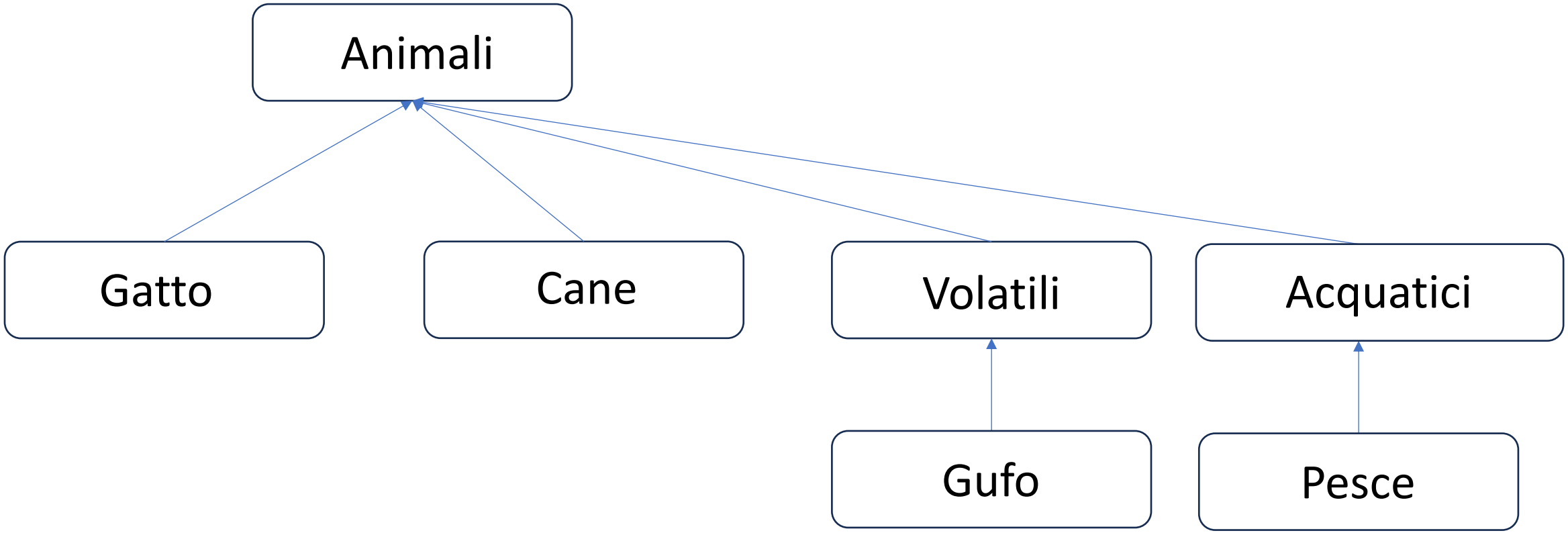
Ereditarietà ed Interfacce

- Consideriamo questa gerarchia

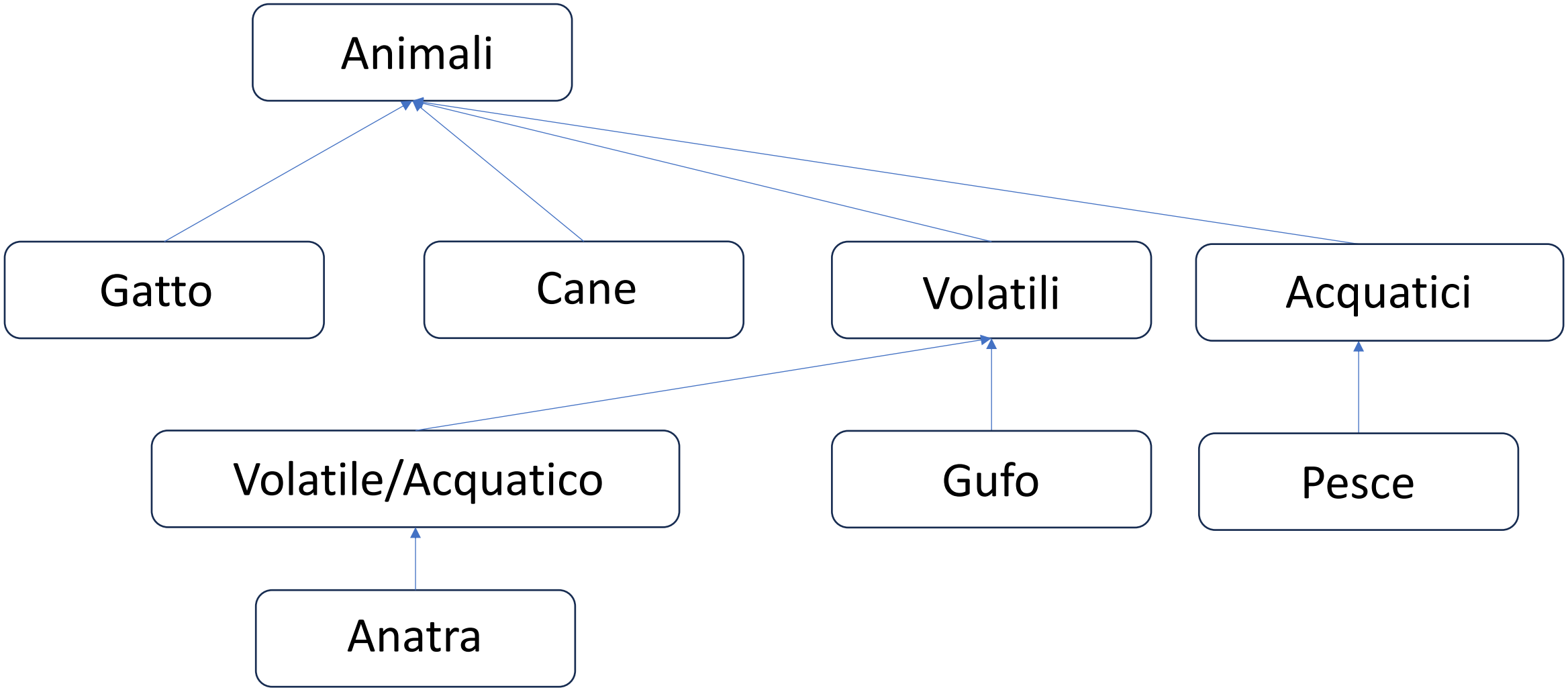


- Ora complichiamola ancora un po’

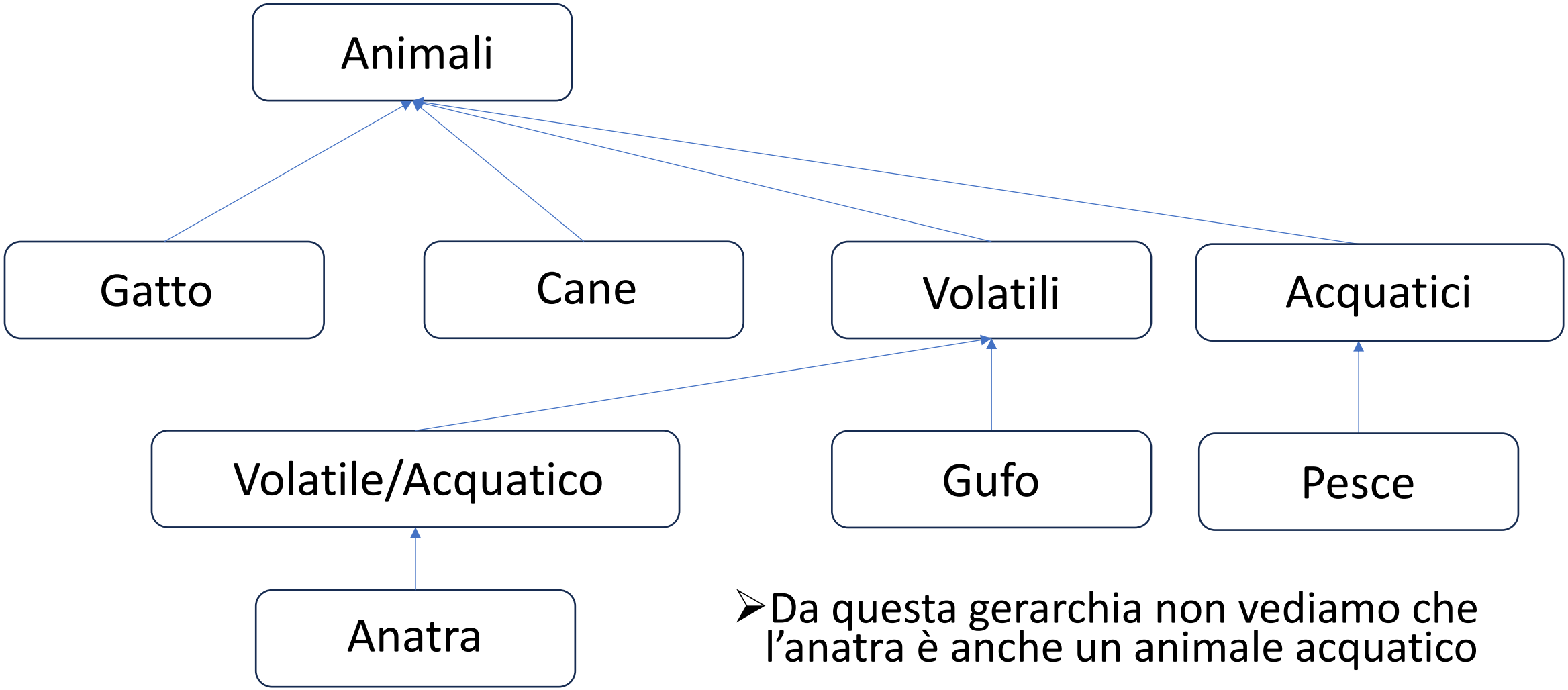
Ereditarietà ed Interfacce



Ereditarietà ed Interfacce



Ereditarietà ed Interfacce



- Da questa gerarchia non vediamo che l'anatra è anche un animale acquatico

Soluzione: Ereditarietà Multipla

- Non è direttamente supportata in Java
  (e.g. potrebbero esserci problemi come il «diamond problem»)
- Può essere realizzata attraverso le **interfacce**

Implementazione

```
1 public class Anatra extends Animale implements Volatile, Acquatico {
2 // Deve implementare i metodi definiti in Volatile ed Acquatico
3 @Override
4 public void vola(){ ... }
5 @Override
6 public void nuota(){ ... }
7 }
```

Classi vs Interfacce

- Una classe rappresenta il fatto che un oggetto è qualcosa.
- Una interfaccia rappresenta il fatto che la classe implementa un certo comportamento.

Interfacce come tipi

- Anche le interfacce possono essere usate come tipo di un riferimento.
- Come tipo statico, un'interfaccia espone tutti i metodi che definisce.
- L'implementazione utilizzata dipende dal tipo dinamico della classe (Binding dinamico).
- Per gli assegnamenti dobbiamo considerare le relazioni di tipo/sotto-tipo.

Attenzione!

- NON CONFONDERE classi astratte ed interfacce

Attenzione!

- NON CONFONDERE classi astratte ed interfacce

1. Una classe può estendere una sola classe astratta, ma può implementare più interfacce.

Attenzione!

- NON CONFONDERE classi astratte ed interfacce

1. Una classe può estendere una sola classe astratta, ma può implementare più interfacce.
2. Una classe astratta può contenere codice e metodi implementati, un’interfaccia NON deve contenere implementazioni.

Attenzione!

- NON CONFONDERE classi astratte ed interfacce

1. Una classe può estendere una sola classe astratta, ma può implementare più interfacce.
2. Una classe astratta può contenere codice e metodi implementati, un’interfaccia NON deve contenere implementazioni.
3. Una variabile dichiarata all’interno di un’interfaccia è di fatto una costante.

Attenzione!

- NON CONFONDERE classi astratte ed interfacce

1. Una classe può estendere una sola classe astratta, ma può implementare più interfacce.
2. Una classe astratta può contenere codice e metodi implementati, un'interfaccia NON deve contenere implementazioni.
3. Una variabile dichiarata all'interno di un'interfaccia è di fatto una costante.
4. I metodi in un'interfaccia sono **sempre** public.

Keyword

implements -> una classe *implementa* un'interfaccia

extends -> una classe *estende* al più una classe.
un'interfaccia può *estendere* una o più interfacce.

Implementazione

```
1 public abstract class Animale {
2 }
```

Implementazione

```
1 public class Gatto extends Animale {
2 }
```

Implementazione

```
1 public class Anatra extends Animale implements Volatile, Acquatico {
2 // Deve implementare i metodi definiti in Volatile ed Acquatico
3 @Override
4 public void vola(){ ... }
5 @Override
6 public void nuota(){ ... }
7 }
```

Quali operazioni sono consentite?

```
1 Animale g = new Gatto();
2 Pesce p = new Pesce();
3 Anatra a = new Anatra();
4 Volatile g1 = g;
5 Volatile v = a;
6 Nuotatore n = a;
7 g.vola();
8 g.nuota();
9 v.vola();
10 n.nuota();
```

Quali operazioni sono consentite?

```
1 Animale g = new Gatto();
2 Pesce p = new Pesce();
3 Anatra a = new Anatra();
4 Volatile g1 = g; //No, g non e' istanza di implementazione di Volatile
5 Volatile v = a;
6 Nuotatore n = a;
7 g.vola(); //No, g non vola
8 g.nuota(); //No, g non nuota
9 v.vola(); //OK
10 n.nuota(); //OK
```

Esercizio Operazioni

Definisci una interfaccia 'operazione' e le sue classi derivate(sub. div, mul) in modo che da ciascuna sia possibile eseguire la corrispondente operazione aritmetica (di addizione, divisione, moltiplicazione) istanziando un oggetto di classe 'operazione' e due operandi. Realizzare anche, una classe astratta che implementi le funzionalità comuni come la stampa a video del risultato.

http://www.edutecnica.it/informatica/abstractx/5.htm

From Private to Public

VARIABLE VISIBILITY

Modificatori di Visibilità

	Classe	Package	Sottoclasse (Stesso package)	Sottoclasse (Package diversi)	Everywhere
Public	✓	✓	✓	✓	✓
Protected					
Package					
Private					

Modificatori di Visibilità

	Classe	Package	Sottoclasse (Stesso package)	Sottoclasse (Package diversi)	Everywhere
Public	✓	✓	✓	✓	✓
Protected	✓	✓	✓	✓	✗
Package					
Private					

Modificatori di Visibilità

	Classe	Package	Sottoclasse (Stesso package)	Sottoclasse (Package diversi)	Everywhere
Public	✓	✓	✓	✓	✓
Protected	✓	✓	✓	✓	✗
Package	✓	✓	✓	✗	✗
Private					

Modificatori di Visibilità

	Classe	Package	Sottoclasse (Stesso package)	Sottoclasse (Package diversi)	Everywhere
Public	✓	✓	✓	✓	✓
Protected	✓	✓	✓	✓	✗
Package	✓	✓	✓	✗	✗
Private	✓	✗	✗	✗	✗

Esercizio Visibilità 1: Modificatori di visibilità:

```
1 package a;
23
..... class First {
4 ..... int x;
5 ..... int y;
67
..... void h() {
8 y = −1;
9 }
10 }
11
12 ..... class Second extends First {
13 ..... void f(int x) {
14 this.x = x;
15 h();
16 }
17 }
```

```
1 package b;
2 import a.∗;
34
class Third {
5 public static void main(String[] s)
{
6 Second z = new Second();
7 z.f(3);
8 }
9
10 class Fourth extends First {
11 void g() {
12 h();
13 }
14 }
```

Esercizio Visibilità 1: Modificatori di visibilità:

```
1 package a;
2
3 public class First {
4 int x;
5 private int y;
67
protected void h() {
8 y = −1;
9 }
10 }
11 public class Second extends First {
12 public void f(int x) {
13 this.x = x;
14 h();
15 }
16 }
```

```
1 package b;
2 import a.∗;
34
class Third {
5 public static void main(String[] s)
{
6 Second z = new Second();
7 z.f(3);
8 }
9
10 class Fourth extends First {
11 void g() {
12 h();
13 }
14 }
```

Modificatori di visibilità

```
1 //Primo file
2 package a;
34
public class C1 {
56
public void m1() { }
7 protected void m2 () { }
8 private void m3 () { }
9 }
```

main

```
1 C2 x = new C2(); //Dire se corretta e
2 x.m1(); //che output produrrebbe
```

```
1 //Secondo file
2 package b;
3 import a.*;
45public class C2 extends C1 {
67
public void m1() {
8 System.out.print("Salve");
9 m2();
10 m3();
11 }
12
13 protected void m2 () {
14 System.out.print(", mondo");
15 }
16
17 private void m3() {
18 System.out.print("!");
19 }
```

Modificatori di visibilità

- La definizione di C2 e’ corretta.
- Il metodo m1 puo’ utilizzare i metodi m2 e m3.
- L’output e’ “Salve, mondo!”.

Modificatori di visibilità

```
1 //Primo file
2 package a;
3
4
public class C1 {
5
6
public void m1() { }
7 protected void m2 () { }
8 private void m3 () { }
9 }
```

```
1 C2 x = new C2(); //Dire se corretta e
2 x.m1(); //che output produrrebbe
```

```
1 package b;
2 import a.*;
3
4 public class C2 extends C1 {
5
6 public void m1() {
7 System.out.print("Salve");
8 m2();
9 m3();
10 }
11
12 protected void m2() {
13 System.out.print(", mondo");
14 }
15
16 //Nessuna definizione di m3
17 }
```

Modificatori di visibilità

- In questo caso C2 non ha visibilità di m3
- Errore di compilazione!!

Modificatori di visibilità

```
1 //Primo file
2 package a;
3
4
public class C1 {
5
6
public void m1() { }
7 protected void m2 () { }
8 private void m3 () { }
9 }
```

```
1 C2 x = new C2(); //Dire se corretta e
2 x.m1(); //che output produrrebbe
```

```
1 package b;
2 import a.*;
3
4public class C2 extends C1 {
5
6
public void m1() {
7 System.out.print("Salve");
8 m2();
9 m3();
10 }
11
12 //Nessuna definizione di m2
13
14 private void m3() {
15 System.out.print("!");
16 }
17 }
```

Modificatori di visibilità

- In questo caso C2 ha visibilità di m2 definito in C1
- L'output sarà *Salve*!

Modificatori di visibilità

```
1 //Primo file
2 package a;
34
public class C1 {
56
public void m1() { }
7 protected void m2 () { }
8 private void m3 () { }
9 }
```

main

```
1 C2 x = new C2(); //Dire se corretta e
2 x.m1(); //che output produrrebbe
```

```
1 //Secondo file
2 package b;
3 import a.*;
45public class C2 extends C1 {
67
public void m1() {
8 System.out.print("Salve");
9 m2();
10 m3();
11 }
12
13 protected void m2 () {
14 System.out.print(", mondo");
15 }
16
17 private void m3() {
18 System.out.print("!");
19 }
```

Esercizio Visibilità 2: Modificatori di visibilità

```
1 package c;
2 import a.*;
3 import b.*;
4 public class C3 {
5 public static void main(String[] s){
6 C1 c1;
7 C2 c2;
8 Object o;
9 c1 = new C1();
10 c1.m1();
11 c2 = new C2();
12 c2.m2();
13 c1 = c2;
14 c1.m1();
15 c2 = new C1();
16 o = new C1();
17 c2 = (C2)o;
18 o = new C2();
19 c1 = (C1) o;
20 c1.m1();
21 }
22 }
```

1. Indicare gli errori a compile-time ed eliminare le istruzioni che li generano
2. Dire se ci sono errori a run-time ed eliminare le istruzioni che li generano
3. Determinare l'output del programma

Esercizio Visibilità 2: Modificatori di visibilità

```
1 package c;
2 import a.*;
3 import b.*;
4 public class C3 {
5 public static void main(String[] s){
6 C1 c1;
7 C2 c2;
8 Object o;
9 c1 = new C1();
10 c1.m1();
11 c2 = new C2();
12 c2.m2(); //m2 protected, errore a tempo di compilazione ( m2 non è visto da c2)
13 c1 = c2;
14 c1.m1();
15 c2 = new C1(); //C1 superclasse di C2, errore a tempo di compilazione
16 o = new C1();
17 c2 = (C2)o; //Errore a run−time: o istanza di C1
18 o = new C2();
19 c1 = (C1) o;
20 c1.m1();
21 }
22 }
```

Esercizio Visibilità 3

Quale di queste operazioni è sbagliata?

```
1 class Persona{
2 String saluto(){
3 return "Buongiorno";
4 }}
5 class PersonaEducata extends Persona
{
6 String saluto() {
7 return "Buongiorno a lei";
8 }}
9 class PersonaMaleducata extends Persona {
10 String saluto() {
11 return "Faccia silenzio!";
12 }}
13 class PersonaMaleducatissima extends PersonaMaleducata
{
14 String saluto() {
15 return "Non mi rompa!";
16 }}
```

```
1 Persona p = new Persona();
2 PersonaEducata pe =new
PersonaEducata();
3 PersonaMaleducata pm =new
PersonaMaleducata();
4 PersonaMaleducatissima pmm=new
PersonaMaleducatissima();
5 p.saluto();
6 pe = p;
7 p = pe;
8 p.saluto();
9 pe = pm;
10 pe.saluto();
11 pm.saluto();
12 p = new PersonaMaleducata();
13 p.saluto();
14 pm = p;
15 pmm = (PersonaMaleducatissima) pm;
16 pmm.saluto();
```

Esercizio Visibilità 3

Quale di queste operazioni è sbagliata?

```
1 class Persona{
2 String saluto(){
3 return "Buongiorno";
4 }}
5 class PersonaEducata extends Persona
{
6 String saluto() {
7 return "Buongiorno a lei";
8 }}
9 class PersonaMaleducata extends Persona {
10 String saluto() {
11 return "Faccia silenzio!";
12 }}
13 class PersonaMaleducatissima extends PersonaMaleducata
{
14 String saluto() {
15 return "Non mi rompa!";
16 }}
```

```
1 Persona p = new Persona();
2 PersonaEducata pe =new
PersonaEducata();
3 PersonaMaleducata pm =new
PersonaMaleducata();
4 PersonaMaleducatissima pmm=new
PersonaMaleducatissima();
5 p.saluto();
6 pe = p;
7 p = pe;
8 p.saluto();
9 pe = pm;
10 pe.saluto();
11 pm.saluto();
12 p = new PersonaMaleducata();
13 p.saluto();
14 pm = p;
15 pmm = (PersonaMaleducatissima) pm;
16 pmm.saluto();
```

Esercizio 4: People & Students

- Implementare una classe per rappresentare Persone e Studenti
- Tener traccia del nome e della data di nascita della persona.
- Gli student sono persone che hanno una lista di esami “tentati”.

Esercizio 4: People & Students

- Implementare una classe per rappresentare Persone e Studenti
- Tener traccia del nome e della data di nascita della persona.
- Gli student sono persone che hanno una lista di esami “tentati”.

HINT: Clone()

clone() è un (protected) method definito in Object per restituire una copia dell’oggetto.

- Per usare questo metodo, la classe deve implementare l’interfaccia Clonable, altrimenti l’implementazione di default lancia soltanto un’eccezione.

- Può essere sostituito da un “copy constructor”.

How to handle the exam

ESERCIZIO D’ESAME: DYNAMIC BINDING

Esercizio 1 - TdE 26/6/2020 (es3)

```
abstract class Person {
        public void join(Lecture lec) {
                System.out.println(”Joining ” + lec); }
        public void join(OnlineLecture lec) {
                System.out.println(”Joining ” + lec); }
}
class Student extends Person {
        public void join(Lecture lec) {
                System.out.println(”Student joining ” + lec); }
}
class Teacher extends Person {
        public void join(OnlineLecture lec) {
                System.out.println(”Teacher joining ” + lec); }
}
class Lecture {
        public void addAttendant(Person p) { p.join(this); }
        public String toString() { return ”a lecture”; }
}
class OnlineLecture extends Lecture {
        public String toString() { return ”an online lecture”; }
}
```

Esercizio 1 - TdE 26/6/2020 (es3)

```
public class Main {
    public static void main( String [] args ) {
1           Person p1 = new Student();
2           Person p2 = new Person();
3           Person p3 = new Teacher();
4           Student p4 = new Student();
5           Student p5 = new Teacher();
6           Lecture lec1 = new Lecture();
7           Lecture lec2 = new OnlineLecture();
8           OnlineLecture lec3 = new OnlineLecture();
                        [...]
```

Esercizio 1 - TdE 26/6/2020 (es3)

```
public class Main {
        public static void main( String [] args ) {
                                        [...]
9               lec1.addAttendant(p1);
10              lec1.addAttendant(p2);
11              lec1.addAttendant(p3);
12              lec1.addAttendant(p4);
13              lec1.addAttendant(p5);
14              lec2.addAttendant(p1);
15              lec2.addAttendant(p2);
16              lec2.addAttendant(p3);
17              lec2.addAttendant(p4);
18              lec2.addAttendant(p5);
19              lec3.addAttendant(p1);
20              lec3.addAttendant(p2);
21              lec3.addAttendant(p3);
22              lec3.addAttendant(p4);
```

Esercizio 1 - TdE 26/6/2020 (es3)

Quesiti:

a) Quali linee del programma non sono valide? Motivare brevemente la risposta.

b) Dopo aver rimosso tutte le linee non valide identificate al punto precedente, indicare cosa stampa il programma, motivando brevemente la risposta.

Esercizio 1 - TdE 26/6/2020 (es3)

```
public class Main {
    public static void main( String [] args ) {
1           Person p1 = new Student();
2           Person p2 = new Person();
3           Person p3 = new Teacher();
4           Student p4 = new Student();
5           Student p5 = new Teacher();
6           Lecture lec1 = new Lecture();
7           Lecture lec2 = new OnlineLecture();
8           OnlineLecture lec3 = new OnlineLecture();
                        [...]
```

Esercizio 1 - TdE 26/6/2020 (es3)

```
public class Main {
    public static void main( String [] args ) {
1           Person p1 = new Student();
2           Person p2 = new Person();
3           Person p3 = new Teacher();
4           Student p4 = new Student();
5           Student p5 = new Teacher();
6           Lecture lec1 = new Lecture();
7           Lecture lec2 = new OnlineLecture();
8           OnlineLecture lec3 = new OnlineLecture();
                        [...]
```

Non si può istanziare una classe astratta

Teacher non è un tipo compatibile con Student

Esercizio 1 - TdE 26/6/2020 (es3)

```
public class Main {
        public static void main( String [] args ) {
                                        [...]
9               lec1.addAttendant(p1);
10              lec1.addAttendant(p2);
11              lec1.addAttendant(p3);
12              lec1.addAttendant(p4);
13              lec1.addAttendant(p5);
14              lec2.addAttendant(p1);
15              lec2.addAttendant(p2);
16              lec2.addAttendant(p3);
17              lec2.addAttendant(p4);
18              lec2.addAttendant(p5);
19              lec3.addAttendant(p1);
20              lec3.addAttendant(p2);
21              lec3.addAttendant(p3);
22              lec3.addAttendant(p4);
```

Esercizio 1 - TdE 26/6/2020 (es3) (soluzione – quesito b)

```
9           lec1.addAttendant(p1);
10          lec1.addAttendant(p2);
11          lec1.addAttendant(p3);
12          lec1.addAttendant(p4);
13          lec1.addAttendant(p5);
14          lec2.addAttendant(p1);
15          lec2.addAttendant(p2);
16          lec2.addAttendant(p3);
17          lec2.addAttendant(p4);
18          lec2.addAttendant(p5);
19          lec3.addAttendant(p1);
20          lec3.addAttendant(p2);
21          lec3.addAttendant(p3);
22          lec3.addAttendant(p4);
23          lec3.addAttendant(p5);
}}
```

p2 e p5 non esistono (di conseguenza)

Esercizio 1 - TdE 26/6/2020 (es3) (soluzione – quesito b)

```
Student joining a lecture
Joining a lecture
Student joining a lecture
Student joining an online lecture
Joning an online lecture
Student joining an online lecture
Student joining an online lecture
Joning an online lecture
Student joining an online lecture
```

`toString()` di `Lecture`

`toString()` viene sovrascritto (override) dinamicamente da `OnlineLecture`

`join(Lecture)` viene sovrascritto (override) dinamicamente da `Student`

Overloading di `join` risolve su `join(Lecture)` staticamente in quanto `this` in `attendLecture` è sempre di tipo `Lecture` staticamente

Esercizio 1 - TdE 26/6/2020 (es3) (soluzione – quesito b)

1 `lec1.addAttendant(p1);`

- **Oggetti coinvolti**: `lec1` è un'istanza di `Lecture`, `p1` è un'istanza di `Student`.
- **Cosa succede**:
  - Il metodo `addAttendant` di `Lecture` viene chiamato su `lec1` con `p1` come argomento.
  - Nel metodo `addAttendant`, si chiama `p.join(this)`, dove `this` è `lec1` (di tipo `Lecture`).
  - Poiché `p1` è un'istanza di `Student`, viene chiamato il metodo `join(Lecture lec)` **sovrascritto in `Student`**.
  - Questo metodo stampa: **`Student joining a lecture`**.
- **Output**:

```
Student joining a lecture
```

Esercizio 1 - TdE 26/6/2020 (es3) (soluzione – quesito b)

2. `lec1.addAttendant(p3);`

- **Oggetti coinvolti**: `lec1` è un'istanza di `Lecture`, `p3` è un'istanza di `Teacher`.
- **Cosa succede**:
  - Il metodo `addAttendant` di `Lecture` viene chiamato su `lec1` con `p3` come argomento.
  - Nel metodo `addAttendant`, si chiama `p.join(this)`, dove `this` è `lec1` (di tipo `Lecture`).
  - Poiché `p3` è un'istanza di `Teacher` e non ha sovrascritto il metodo `join(Lecture lec)`, viene utilizzato il metodo ereditato dalla classe `Person`.
  - Il metodo ereditato stampa: **`Joining a lecture`**.
- **Output**:

```
Joining a lecture
```

Esercizio 1 - TdE 26/6/2020 (es3) (soluzione – quesito b)

3. `lec1.addAttendant(p4);`

- **Oggetti coinvolti**: `lec1` è un'istanza di `Lecture`, `p4` è un'istanza di `Student`.
- **Cosa succede**:
  - Il metodo `addAttendant` di `Lecture` viene chiamato su `lec1` con `p4` come argomento.
  - Nel metodo `addAttendant`, si chiama `p.join(this)`, dove `this` è `lec1` (di tipo `Lecture`).
  - Poiché `p4` è un'istanza di `Student`, viene chiamato il metodo `join(Lecture lec)` **sovrascritto in `Student`**.
  - Questo metodo stampa: **`Student joining a lecture`**.
- **Output**:

```
Student joining a lecture
```

Esercizio 1 - TdE 26/6/2020 (es3) (soluzione – quesito b)

4. `lec2.addAttendant(p1);`

- **Oggetti coinvolti**: `lec2` è un'istanza di `OnlineLecture`, `p1` è un'istanza di `Student`.
- **Cosa succede**:
  - Il metodo `addAttendant` di `Lecture` viene chiamato su `lec2` con `p1` come argomento (poiché `OnlineLecture` eredita `addAttendant` da `Lecture`).
  - Nel metodo `addAttendant`, si chiama `p.join(this)`, dove `this` è `lec2` (implementata come `OnlineLecture`, ma il tipo statico di this è Lecture, dato che il metodo è ereditato da Lecture).
  - Poiché `p1` è un'istanza di `Student`, viene chiamato il metodo `join(Lecture lec)` **sovrascritto in `Student`**.
  - Questo metodo stampa: **`Student joining an online lecture`**.
- **Output**:

```
Student joining an online lecture
```

Esercizio 1 - TdE 26/6/2020 (es3) (soluzione – quesito b)

5. `lec2.addAttendant(p3);`

- **Oggetti coinvolti**: `lec2` è un'istanza di `OnlineLecture`, `p3` è un'istanza di `Teacher`.
- **Cosa succede**:
  - Il metodo `addAttendant` di `Lecture` viene chiamato su `lec2` con `p3` come argomento.
  - Nel metodo `addAttendant`, si chiama `p.join(this)`, dove `this` è `lec2` (di tipo `OnlineLecture` ma il tipo statico di this è Lecture, dato che il metodo è ereditato da Lecture).
  - Poiché `p3` è un'istanza di `Teacher` e il metodo `join(Lecture lec)` non è stato sovrascritto in `Teacher`, viene utilizzato il metodo ereditato dalla classe `Person`.
  - Il metodo ereditato stampa: **`Joining an online lecture`**.
- **Output**:

```
Joining an online lecture
```

Esercizio 1 - TdE 26/6/2020 (es3) (soluzione – quesito b)

6. `lec2.addAttendant(p4);`

- **Oggetti coinvolti**: `lec2` è un'istanza di `OnlineLecture`, `p4` è un'istanza di `Student`.
- **Cosa succede**:
  - Il metodo `addAttendant` di `Lecture` viene chiamato su `lec2` con `p4` come argomento.
  - Nel metodo `addAttendant`, si chiama `p.join(this)`, dove `this` è `lec2` (di tipo `OnlineLecture`).
  - Poiché `p4` è un'istanza di `Student`, viene chiamato il metodo `join(Lecture lec)` **sovrascritto in `Student`**.
  - Questo metodo stampa: **`Student joining an online lecture`**.
- **Output**:

```
Student joining an online lecture
```

Esercizio 1 - TdE 26/6/2020 (es3) (soluzione – quesito b)

7. `lec3.addAttendant(p1);`

- **Oggetti coinvolti**: `lec3` è un'istanza di `OnlineLecture`, `p1` è un'istanza di `Student`.
- **Cosa succede**:
  - Il metodo `addAttendant` di `Lecture` viene chiamato su `lec3` con `p1` come argomento.
  - Nel metodo `addAttendant`, si chiama `p.join(this)`, dove `this` è `lec3` (di tipo `OnlineLecture`).
  - Poiché `p1` è un'istanza di `Student`, viene chiamato il metodo `join(Lecture lec)` **sovrascritto in `Student`**.
  - Questo metodo stampa: **`Student joining an online lecture.`**
- **Output**:

```
Student joining an online lecture
```

Esercizio 1 - TdE 26/6/2020 (es3) (soluzione – quesito b)

8. `lec3.addAttendant(p3);`

- **Oggetti coinvolti**: `lec3` è un'istanza di `OnlineLecture`, `p3` è un'istanza di `Teacher`.
- **Cosa succede**:
  - Il metodo `addAttendant` di `Lecture` viene chiamato su `lec3` con `p3` come argomento.
  - Nel metodo `addAttendant`, si chiama `p.join(this)`, dove `this` è `lec3` (di tipo `OnlineLecture`, ma il tipo statico di this è Lecture, dato che il metodo è ereditato da Lecture).
  - Poiché `p3` è un'istanza di `Teacher` e il metodo `join(Lecture lec)` non è stato sovrascritto in `Teacher`, viene utilizzato il metodo ereditato dalla classe `Person`.
  - Il metodo ereditato stampa: **`Joining an online lecture`**.
- **Output**:

```
Joining an online lecture
```

Esercizio 1 - TdE 26/6/2020 (es3) (soluzione – quesito b)

9. `lec3.addAttendant(p4);`

- **Oggetti coinvolti**: `lec3` è un'istanza di `OnlineLecture`, `p4` è un'istanza di `Student`.
- **Cosa succede**:
  - Il metodo `addAttendant` di `Lecture` viene chiamato su `lec3` con `p4` come argomento.
  - Nel metodo `addAttendant`, si chiama `p.join(this)`, dove `this` è `lec3` (di tipo `OnlineLecture`).
  - Poiché `p4` è un'istanza di `Student`, viene chiamato il metodo `join(Lecture lec)` **sovrascritto in `Student`**.
  - Questo metodo stampa: **`Student joining an online lecture`**.
- **Output**:

```
Student joining an online lecture
```

New

Utilizzo di This

- Il **tipo statico** di this è sempre il **tipo della classe in cui il codice è scritto**.
- Il **tipo dinamico** di this è il **tipo concreto dell'oggetto sul quale il metodo viene invocato a runtime**.

```
public class B{
        void print(){
                System.out.println(this.getClass().getName())
}
}
public class A extends B{
}
```

Interfaccia Iterator <E>

- L'interfaccia Iterator <E> permette di scandire e rimuovere oggetti da una collezione.
- E' composta dai metodi:

```
1 boolean hasNext()
2 E next();
3 void remove();
```

- Esempio

```
1 Iterator<String> iterator = list.iterator();
2 while(iteratore.hasNext()){
3 string = iterator.next();//Elemento successivo
4 iterator.remove();//Rimuovi elemento corrente
5 }
```

Interfaccia Iterable <T>

- L’interfaccia Iterable<T> permette di implementare il for generalizzato (for each).
- E’ composta dal metodo:

Iterator<T> iterator();

- E’ possibile usare il for generalizzato (for each) su oggetti che implementano tale interfaccia.

Exercise Iterable

Si vuole implementare una struttura dati **Stack** (pila LIFO – *Last In, First Out*) generica in Java, utilizzando internamente una ArrayList per la memorizzazione degli elementi.

Requisiti
Definire una classe generica Stack<E> che implementi l'interfaccia Iterable<E>.
All'interno della classe, dichiarare un campo privato di tipo ArrayList<E> che conterrà gli elementi dello stack.

Implementare i seguenti metodi pubblici:

- **push(E e)**: inserisce un elemento nello stack.
- **pop()**: rimuove e restituisce l'elemento in cima allo stack. Se lo stack è vuoto, deve sollevare una specifica eccezione personalizzata (ad esempio NoElementException).
- **isEmpty()**: restituisce true se lo stack è vuoto, false altrimenti.
- **pushAllIn(Stack<? super E> other)**: inserisce tutti gli elementi di *questo* stack nello stack ricevuto come parametro (attenzione all'uso di wildcard con super).
- **pushAll(Stack<? extends E> other)**: inserisce in *questo* stack tutti gli elementi contenuti nello stack ricevuto come parametro (attenzione all'uso di wildcard con extends).

Exercise Iterable

Implementare una **classe interna** StackIterator che realizza l'interfaccia Iterator<E>.

- L'iteratore deve permettere di scorrere lo stack dall'alto verso il basso (cioè dall'ultimo elemento inserito fino al primo).
- Il metodo next() deve sollevare una NoSuchElementException se non ci sono più elementi.

Il metodo iterator() della classe Stack deve restituire un nuovo oggetto StackIterator.

Note

La classe deve essere completamente generica (Stack<E>).

Lo stack deve rispettare il comportamento **LIFO**.

Per la gestione di errori in pop(), definire una propria eccezione controllata o non controllata (a scelta), che incapsuli eventualmente un'altra eccezione (IndexOutOfBoundsException).

To enjoy your free time

EXTRA MATERIAL

Esercizio 5: Complex Number

- Fare una classe per gestire i numeri complessi(class Complex).
- Che funzionalità dovrebbe offrire?

Esercizio 5: Complex Number

- **Operations**:
  - Sum, difference
  - Equality (given the requested precision)
  - Modulo, phase
  - Complex conjugate
  - n-th radix

Esercizio 5: Complex Number

- **Representation**:

  - Parte reale e parte immaginaria

Esercizio 5: Complex Number

- **Representation**:

  - Parte reale e parte immaginaria

- **Attributes**:

  - re, im (double); Rappresentano parte reale ed immaginaria, rispettivamente

Esercizio 5: Complex Number

- **Representation**:

  - Parte reale e parte immaginaria

- **Attributes**:

  - re, im (double); Rappresentano parte reale ed immaginaria, rispettivamente

- **Methods**

  - Constructors: inizializza l'oggetto.

  - Observers: restituisce l'informazione sullo stato dell'oggetto.

  - Producers: restituisce nuovi oggetti.

Esercizio 5: Complex Number

- **Representation**:

  - Parte reale e parte immaginaria

- **Attributes**:

  - re, im (double); Rappresentano parte reale ed immaginaria, rispettivamente

- **Methods**

  - Constructors: inizializza l'oggetto.

  - Observers: restituisce l'informazione sullo stato dell'oggetto.

  - Producers: restituisce nuovi oggetti.

- The class should be ***Immutable***

  - Non deve essere possible modificare lo stato dell'oggetto **dopo** l'instanziazione.

  - Nessun metodo deve poter modificare lo stato dell'oggetto dopo l'instanziazione.

Class Complex

```
1 public class Complex {
2         private final double re, im;
3         public Complex() { ... }
4         public Complex(double re, double im) { ... }
5         public double re() { ... }
6         public static Complex fromPolarForm(double abs, double phase) { ... }
7         public double im() { ... }
8         public double abs() { ... }
9         public double phase() { ... }
10        public Complex conjugate() { ... }
11        public Complex sum(Complex other) { ... }
12        public Complex diff(Complex other) { ... }
13        public Complex mult(Complex other) { ... }
14         public boolean almostEquals(Complex other, double precision) { ... }
15         public ComplexSet getRoots(int degree) { ... }
16        }
```

Esercizio GeometricShape

Implementare una interfaccia TwoDimensionalCanvas che dichiari i metodi implementati dalla classe BasicCanvas, fornita nel materiale didattico. Inoltre, definire una classe astratta Shape, e due sue estensioni, Square e Circle, data la classe Point, definita nel materiale didattico.

http://www.edutecnica.it/informatica/abstractx/5.htm

Torniamo alle forme geometriche

- Si modifichi l'implementazione delle forme geometriche per permettere di "cambiare aspetto"
- Si definiscano opportunamente le interfacce ***Rotatable*** e ***Resizable***

Interfacce Rotatable e Resizable

```
1 public interface Rotatable {
2 Shape rotate(double angle);
3 }
4
5 public interface Resizable {
6 Shape resize(double scale);
7 }
```

- Una forma modificata è ancora una forma
- Il tipo di ritorno è un generico *Shape*